

Complete Architecture Analysis: Branch N vs Branch M

Branch N: cursor/transaction-step-dispatch-4c01

Branch M: cursor/package-layout-refactor-a435

Repository: essaylinkrefactor

Type: Senior architectural review (not a Git diff)

1. Branch N - Architecture

Narrative

N introduces PosTerminal as a reactive step dispatcher and EmvEngine as an abstract lifecycle (prepare / execute / complete). Host authorize and print live on PosTerminal.handleOnlineRequired / handleApproved.

But the composition root still builds the old stack (EmvTransaction + EmvBehavior + runners), while the UI already expects PosTerminal. So N is two architectures at once.

High-level flow

```
UI (MainActivity / MainViewModel)
  |
  | expects getTerminal()
  v
Intended: PosTerminal -> EmvEngine(prepare/execute/complete)
              -> CommunicationBehavior / PrinterBehavior

Actually wired: EmvEnhanceApp -> EmvTransaction
                          -> EmvBehavior
                          -> Contact/Contactless Runners
                          -> Simple* Callback Adapters
```

Module relationship

```
:app (UI + vendors)
-> :core (ports + dual stacks)
-> :emvflow (runners/adapters)
-> PAX stack + WMRouter
```

Card search in N

Not a first-class port. Folded into EmvBehavior / runners / preprocess. PosTerminal does not search.

Strengths (N)

- Clear idea: sticky transaction stream + fire-and-forget EMV stream
- Host/print side-effects centralized on PosTerminal handlers
- Open/Closed intent via overridable handle* methods

Weaknesses (N)

- Dual unfinished stacks (largest issue)
- App/UI seam broken (getTransaction vs getTerminal)
- PaxEmvEngine stub; real PAX still on runners
- Fat unused handleEmv* surface
- Card search not vendor-portable as a contract
- README documents types that do not exist

2. Branch M - Architecture

Narrative

M completes a single coherent story:

- UI talks only to PosTerminal.
- Terminal owns card search + host + printer.
- EmvBehavior owns EMV after entry selection.
- EmvEngine is a thin event bus.

High-level flow

```
UI -> PosTerminal.acceptCard() / startTransaction()
    +-- EmvBehavior.prepare(engine, config)
    +-- PosTerminal.searchCard(config, listener)
    +-- EmvBehavior.start(engine, config, card)

EmvEngine - thin subjects + notify* -> EmvBehavior.dispatch*
```

Package organization

core (com.emvenhance.core)

Package	Contents
`terminal`	PosTerminal, EmvBehavior
`engine`	EmvEngine
`card`	EntryMethod, CardPresence, CardSearchListener, TransactionConfig
`event`	TransactionStep(Event), EmvStep(Event)
`host`	CommunicationBehavior, PrinterBehavior, AuthResult, HostDefaults

app vendors

```
TerminalFactory.create(VENDOR)
PAX      -> PaxTerminal -> PaxEmvBehavior
INGENICO -> IngenicoTerminal -> IngenicoEmvBehavior (stub)
FAKE     -> FakeTerminal -> FakeEmvBehavior
```

emvflow

runtime - preprocess - progress - device - pin

Card search in M

Owner: PosTerminal via searchCard + CardSearchListener.

Events: started - chip - contactless - mag - manual - removed - timeout - cancelled - error.

Object ownership (M)

```
PosTerminal owns:
+-- EmvEngine
+-- EmvBehavior
+-- CommunicationBehavior -> authorize()
+-- PrinterBehavior       -> printReceipt()
```

Strengths (M)

- One public API; UI/core never import vendors
- Card search is a real, portable contract
- Engine is genuinely thin
- Host/printer owned by terminal (correct layer)
- Package boundaries match responsibilities
- Adding a vendor is mechanical
- Dual stacks / runners / callback adapters removed from app path

Weaknesses (M)

- PAX runtime/WMRouter still under the hood
- FAKE/INGENICO Gradle still pull PAX modules
- PaxEmvBehavior is a large multi-interface class
- Ingenico is structural stub, not a second real SDK proof
- Async DAL race on first PAX transaction
- Host still passed into behaviors for online
- Unused CardSearchListener.Adapter

3. Side-by-side Comparison

Category	N	M	Winner	Why
Overall architecture	4/10	9/10	M	N is two incomplete stacks; M is one coherent flow
Separation of concerns	5/10	9/10	M	Search/host/printer/EMV/engine cleanly split in M
Module boundaries	6/10	8/10	M	Core packages in M; both still Gradle-coupled to PAX
Package organization	4/10	9/10	M	N flat + dual models; M intentional packages
Vendor independence	4/10	8/10	M	N folds search into engine/runners; M has searchCard + factory
Extensibility	5/10	9/10	M	Add vendor without touching engine/UI in M
Scalability	5/10	8/10	M	M scales by vendor plug-ins
SOLID	4/10	8/10	M	N violates ISP + unfinished DIP composition
Event architecture	7/10	8/10	M	Same dual-stream idea; M completes wiring
Transaction lifecycle	5/10	9/10	M	Clear prepare->search->start
Card search design	3/10	9/10	M	N has a port in N; first-class in M
Host/printer placement	7/10	9/10	M	M finishes ownership without AbstractEmvBehavior

Simplicity	3/10	8/10	M	N has EmvTransaction + EmvEngine + runners + adapters
Readability	4/10	8/10	M	M's mental model fits one diagram
Testability	5/10	8/10	M	FakeTerminal isolates without runners
Performance design	6/10	7/10	M	Similar IO-thread blocking; M removes adapter hops
Maintainability	3/10	8/10	M	N mid-migration is high risk
Composition root	2/10	9/10	M	N App/UI mismatch is an architectural break
Weighted overall	**~4.5**	**~8.3**	**M**	

Where N is better: The dual event-stream concept and "terminal owns side effects" originated clearly in N.

Where M is superior: Nearly everything else - composition completeness, card-search abstraction, removal of dual stacks.

4. Best-of-Both Recommendation

Do not ship N. Take M as the base, keep N's best ideas, cut remaining debt.

Rules

1. One stack only - prepare -> search -> start
2. Keep dual event streams (sticky transaction + publish EMV)
3. Card search on PosTerminal
4. Host/printer on PosTerminal
5. No AbstractEmvBehavior auto online/print
6. No empty handler forest
7. Vendor = Terminal + EmvBehavior + factory line
8. Gradle flavors - PAX modules only on PAX flavor
9. Online via terminal.authorize(config) long-term
10. Print as UI/terminal policy, not inside EmvBehavior
11. Delete unused Adapter unless needed
12. Prove second vendor with real SDK

Final verdict

Question	Answer
Superior overall?	**Branch M** , clearly
Is N worthless?	No - dual events + terminal side-effects
Merge N as-is?	**No**
Production base?	**M** + flavor isolation + real second vendor
Biggest M risk?	PAX SDK/runtime coupling, unproven Ingenico

Bottom line: Branch N sketched the reactive shell but left two competing designs. Branch M finished the architecture around PosTerminal as the single vendor-agnostic API. Ideal = M's structure + N's event insight ? leftover PAX compile coupling ? unused ceremony.